

mksglu/context-mode

Context Mode : un serveur MCP et des hooks multi-plateformes, 255 fichiers de tests vitest, 119 issues ouvertes dont 4 bugs étiquetés

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

21624 ★

1555 forks

226 issues ouvertes

Elastic License 2.0 (ELv2)

<https://github.com/mksglu/context-mode>

Que fait ce projet et quels sont ses parcours critiques ?

Context Mode est un serveur MCP plus des hooks qui interceptent les sorties d'outils des agents de code, les indexent dans SQLite FTS5, et n'exposent qu'un résumé recherchable ; il persiste aussi la session pour survivre à la compaction et impose un routage sur 17 plateformes ^{1 2}.

Parcours critiques déduits du README et de CONTRIBUTING.md : - Interception d'une sortie d'outil, indexation, recherche via `ctx_search` ³.
- Exécution de code en bac à sable dans 12 langages (`src/executor.ts`) et règles deny/allow (`src/security.ts`) ³. - Compaction puis reprise : hooks `PreCompact`, `SessionStart`, snapshot injecté ³. - Installation et diagnostic par la CLI (`setup` , `doctor`) sur chaque plateforme ⁴. Le repo compte 226 issues ouvertes toutes catégories ⁵.

Comment est-il testé aujourd'hui ?

`.agents/` config agents IA

`.claude-plugin/` manifeste plugin Claude Code

`.claude/` config locale Claude Code

`.codex-plugin/` manifeste plugin Codex

`.cursor-plugin/` manifeste plugin Cursor

`.gitattributes` attributs git

`.github/` 5 workflows GitHub Actions

`.gitignore` exclusions git

`.mcp.json.codex.example` exemple config MCP Codex

`.mcp.json.example` exemple config MCP

`.npmignore` exclusions publication npm

`.openclaw-plugin/` manifeste plugin OpenClaw

`.pi/` extension pour Pi

`BENCHMARK.md` résultats de benchmark

`CLAUDE.md` instructions pour Claude Code

`CONTRIBUTING.md` guide de contribution

`LICENSE` licence Elastic 2.0

`README.md` documentation principale

`bin/` script statusline.mjs

<code>bun.lock</code>	verrou dépendances bun
<code>cli.bundle.mjs</code>	CLI bundlée par esbuild
<code>configs/</code>	configs par plateforme
<code>docs/</code>	documentation
<code>hooks/</code>	hooks par plateforme, bundles session
<code>openclaw.plugin.json</code>	manifeste plugin OpenClaw
<code>package.json</code>	manifeste npm
<code>scripts/</code>	scripts build, install, version
<code>server.bundle.mjs</code>	serveur MCP bundlé par esbuild
<code>skills/</code>	skills pour agents
<code>src/</code>	TypeScript : serveur MCP, CLI, adaptateurs
<code>start.mjs</code>	lanceur publié avec le paquet
<code>stats.json</code>	statistiques publiées
<code>tests/</code>	tests vitest
<code>tsconfig.json</code>	config TypeScript
<code>vitest.config.ts</code>	config vitest
<code>web/</code>	partie web du projet

Framework vitest, dossier `tests/` , 255 fichiers trouvés par recherche de code ^{6 7 8}. `npm test` lance `vitest run` , précédé automatiquement d'un `npm run build` ^{9 4}.

Les 20 premiers chemins montrent des sous-dossiers `tests/executor/` , `tests/core/` (deny-policy, routing), `tests/session/` , `tests/adapters/` (kiro, memory-conventions) et `tests/util/` , plus un `tests/mcp-integration.ts` ¹⁰. Un benchmark existe : `npx tsx`

tests/benchmark.ts⁴.

Le cache ne contient ni couverture de code ni liste complète des 255 fichiers ; à vérifier dans <https://github.com/mksglu/context-mode/tree/main/tests>.

Que dit la CI et quand tourne-t-elle ?

Un workflow lu, CI (`.github/workflows/ci.yml`), déclenché par `workflow_dispatch`, `push` et `pull_request`, sur les branches `main` et `next` ¹¹
¹² ¹³.

Matrice ubuntu, macos et windows, fail-fast désactivé ; Node 22.5, Python 3.12, Go stable, Elixir 1.17 installés pour l'exécuteur polyglotte ¹³.
Étapes : `npm install`, `typecheck`, `build`, `bundle`, assertions sur les bundles (garde-fou issue #511), puis `vitest` et enfin `npx tsx src/cli.ts`
`doctor` en `continue-on-error` ¹³.

Quatre workflows non lus : `bundle`, `openclaw-e2e`, `tier2-e2e-smoke`, `update-stats` ¹³. Les deux e2e sont à examiner pour connaître la couverture bout en bout réelle.

Où sont les bugs connus ?



119 issues ouvertes, 6 fermées sur 30 jours ; seules 11 sont étiquetées : 6 enhancement, 4 bug, 1 help wanted ^{14 15 16}. Le tri par label est donc peu fiable, lire les titres.

Bugs les plus discutés : - #950 : `ctx_stats` se contredit dans un même rendu et annonce 100 % d'économie sans aucun appel ^{17 18}. - #947 : le timeout de `ctx_batch_execute` ne borne pas l'indexation, un agent peut pendre des heures ¹⁹. - #959 : sur l'adaptateur Pi, un `ctx_execute` bloqué ne peut pas être interrompu ²⁰. - #1024 : le nettoyage des bases périmées supprime des sessions vivantes mais inactives ²¹. - #1022 : le snapshot de reprise ignore son budget d'octets, ~196 KB injectés au lieu de 2 KB ²². - #911 et #946 : l'injection de prompt déclenche le classificateur de Claude Code ^{23 24}.

Comment reproduire et signaler un bug ici ?

Le mainteneur demande trois choses : suivre les templates d'issue, lancer `bash scripts/ctx-debug.sh` et joindre sa sortie, écrire des tests pour tout correctif ³. Pour tester localement dans une vraie session : `npm run build` puis supprimer `server.bundle.mjs` , sinon vos changements ne sont jamais chargés ³.

Côté PR : 107 ouvertes, aucune fusionnée sur 30 jours dans le cache ²⁵ ²⁶. Attendez-vous à une revue lente ; un rapport de bug précis avec sortie de debug a plus de valeur qu'une PR non testée.

Quelles zones sont peu couvertes ?

Le cache ne contient pas de mesure de couverture ; les zones ci-dessous sont déduites des issues et de la structure, à vérifier dans <https://github.com/mksglu/context-mode/tree/main/tests>. - Timeouts et annulation (#947, #959) : les tests vus portent sur l'exécuteur et le bac à sable Windows, pas sur l'interruption ^{19 10}. - Budgets et statistiques (#950, #1022) : `tests/session/stats-output-format.test.ts` existe mais les bugs persistent ^{10 17}. - Plateformes secondaires : issue #45 cherche encore des testeurs sur 15 plateformes × 3 OS ²⁷. Les adaptateurs kiro et memory-conventions ont des tests, les autres n'apparaissent pas dans les 20 chemins vus ¹⁰. - Activité concentrée : 60 des commits sur 12 semaines viennent d'une seule personne, donc peu de relecture croisée ^{28 29}.

Plan de test en 5 points

1. Installer : `npm install`
2. Lancer les tests : `npm test`
3. Ouvrir `src/server.ts` — Serveur MCP : script dev = `npx tsx src/server.ts`, bundlé en `server.bundle.mjs` par esbuild (package.json).

Sources

1. donnée collectée `repo.description`
2. source du repo `readme.md`
3. source du repo `contributing.md`
4. source du repo `manifest.md`
5. donnée collectée `repo.open_issues`
6. donnée collectée `tests.framework`
7. donnée collectée `tests.dir`
8. donnée collectée `tests.files`
9. donnée collectée `build.test`
10. source du repo `tests.md`
11. donnée collectée `build.ci.0.name`
12. donnée collectée `build.ci.0.triggers`
13. source du repo `ci.md`
14. donnée collectée `issues.open`
15. donnée collectée `issues.closed_30d`
16. donnée collectée `issues.by_label`
17. donnée collectée `issues.hot.3.url`
18. donnée collectée `issues.hot.3.comments`
19. donnée collectée `issues.hot.4.url`

- 20. donnée collectée `issues.hot.2.url`
- 21. donnée collectée `issues.hot.8.url`
- 22. donnée collectée `issues.hot.9.url`
- 23. donnée collectée `issues.hot.1.url`
- 24. donnée collectée `issues.hot.5.url`
- 25. donnée collectée `pulls.open`
- 26. donnée collectée `pulls.merged_30d`
- 27. donnée collectée `issues.hot.0.title`
- 28. donnée collectée `activity.contributors.0.commits`
- 29. donnée collectée `activity.bus_factor`